Project: IR Communication

CSE3442

Spring 2025

Won Heo, #1002233878

Plan and Design

SECTION

0. Introduction
1. Block diagram
2. Software Approach
3. Hardware Checkout

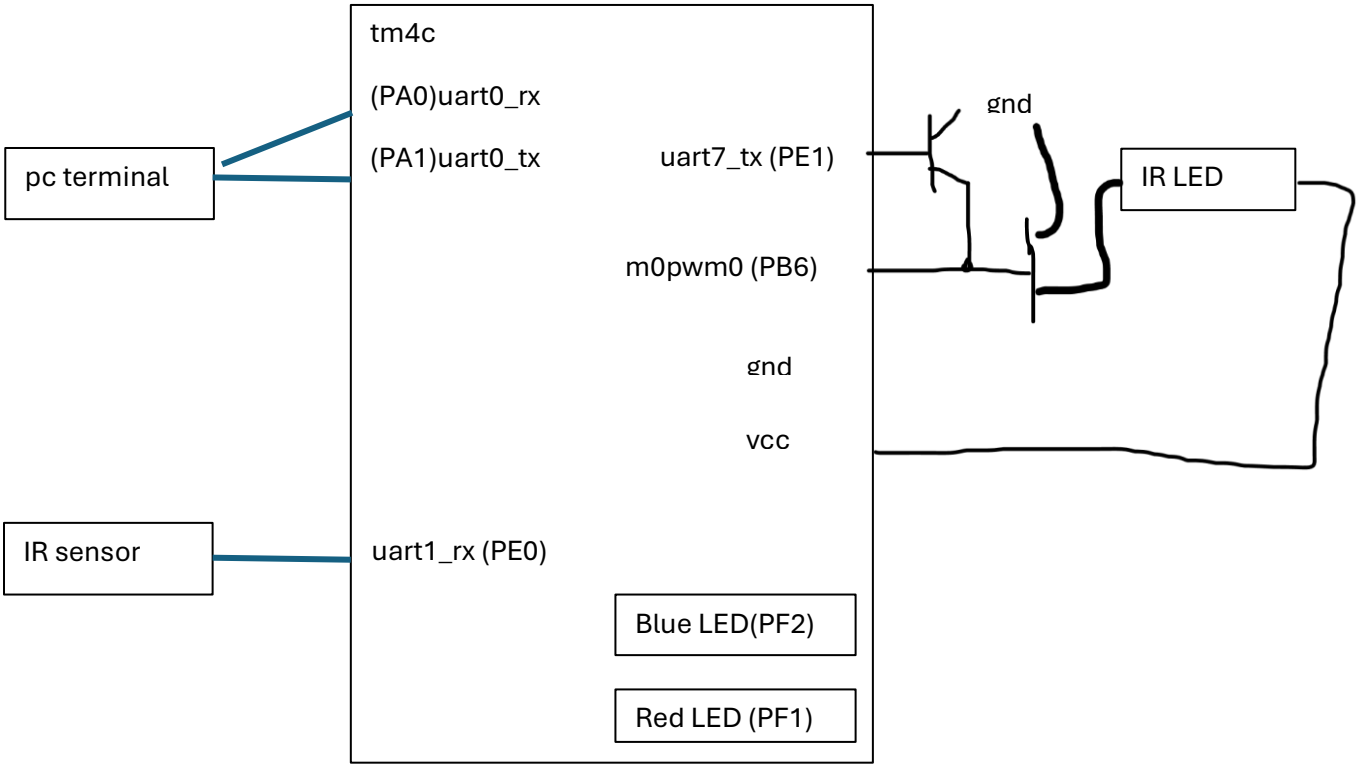
0. Introduction

The objective of this project is to design and implement a full-duplex IR communication link between two TM4C123GXL microcontrollers. Optical shield will be used to isolate each sensor from the emitter, preventing interference.

To ensure compatibility with TSOP134 receiver, **PWM** will be used to generate 38kHz carrier signal. The serial data transmission (300 baud, 8E1) is managed in the **foreground** (main loop), while data reception is handled via **background** ISRs to ensure non-block processing.

In addition to the basic requirements, this design incorporates specific goal requirements for diagnostics: **Error Handling**: Serial data errors (Parity and Framing) will be identified and communicated to user via **uart0**. Additionally, on-board **Red LED** will illuminated for 1sec upon error detection. ***ISR Profiling**: **Blue LED** will illuminated upon entering sensor receive ISR.

1. Block diagram



3. Software approach

3.1 Foreground Process (main)

main() is responsible for Initialization, Command Parsing, and low-priority data transmission.

1. Initialization:
   > config **UART0** for 115200 baud, 8N1 format for **PC terminal**.
   > config **UART1** for 300 baud, 8E1 format for **IR Link**.
   > config **PWM** to generate 38k carrier signal output.
   > config on-board **RED LED** for **data errors**.
   > config **BLUE LED** on entering an **UART1_RX ISR**

2. Main while loop:
   > Check PC Command Flag -> Process Command -> UART1 TX buffer -> UART1 interrupt
   (loop will poll the TX_fifo status register to send one byte at a time without blocking)

   > Check IR Flag (UART1 RX ISR) -> Read data

Buffer and its pointers are shared between ISR and Main, and they are volatile.

3.2 Background Process (interrupt)

Time-critical data movement (filling /emptying HW FIFOs), toggling blue led

1. **UART0_RX** interrupt (PC terminal -> uart0_rx)
   > triggered when FIFO is not empty
   > Interrupt **Priority 2**, fastest in our interrupts, to prevent overrun at 115200 baud

2. **UART1_RX** interrupt (IR sensor -> uart1_rx)
   UART1_TX interrupt (pc_data_buffer -> uart1_tx)
   > triggered when FIFO is not empty
   > illuminate **BLUE LED** upon ISR entry
   (LED illuminate only for short time, not visible through human eyes.
   I'll use osilioscope to detect the HIGH(1). It's to confirm we are getting data through IR SENSOR)

   > Interrupt **Priority 3**, slower than uart0 300baud

3. **TIMER0A**_interrupt (upon uart1rx PE/ FE error)
   > check UART Status Register for **Parity errors/ Framing Errors**, and set **flags, RED_LED for 1second.**

   > one shot timer

   > Interrupt **Priority 5,** not critical in our system.

3.3 Timing Analysis

> Uart0 (115200 baud, 8N1 10bits) takes **87us** to send one character and takes **1.39ms** to fill up 16x8 FIFO.

-> 115200^-1 * 10 = 87us

-> 115200^-1 * (10 * 16) = 1.39ms

> IR Communication (300 baud, 8E1 11bits) takes 33.3ms to send one character.
-> 300^-1 * 11 = 36.6ms
-> 300 ^-1 * (11* 16) = 586ms

Conflict 1: PC sends 16bytes in 1.39ms. IR takes 586ms to re-transmit this. Therefore, blocking implementation would freeze CPU for 0.5 seconds. This is why we need **interrupt driven** design.

Conflict 2: hardware FIFO constraint to 16bytes deep, but data is upto 20bytes. We need **software FIFO buffer** for this.

4. Hardware Checkout

1. Check **38khz carrier** output (m0pwm0, PB6)
   Connect Oscilloscope on PWM output Pin and verify signal.

2. Check **IR data output** (uart1_tx, PB1)
   High on Idle, Low on data start

capture falling edge when data sent to uart1_tx

3. Check **transistors** Connection work

Uart (transmit)	pwm	IR led	IR sensor	Uart (receive)
0	0	0	1	1
1	0	0	1	1
0	1	1	0	0
1	1	0	1	1

4. Check **IR Sensor** Receives data
   Connect oscilloscope on IR Sensor output leg and check if receives data upon IR LED blinks.

4.2 Test cases

1. Buffer wrap test: send 21bytes (20 + @) to ensure Software FIFO (Ring Buffer) wraps around correctly without overwriting unread data.

2. Error injection: Cause Framing Error by blocking IR Sensor/ Cause Parity Error by send command “Change Parity”. Verify Red LED stays on for exactly 1 second.

3. Full Duplex test: Send data from PC while IR Sensor receive and send data simultaneously. Verify no characters are lost